

हेलो! आपका स्वागत है **Unit 19: Modules** की complete study class में.

मैंने आपके workspace में, unit 19 modules folder के अंदर ये complete study files write कर दी हैं:

- Study Guide File: chapter19_modules.md
- Module Declaration File: math.js
- Main Entry Module: app.js

चलिए अब Modules, scope isolation namespaces, named exports vs default exports aur browser level HTML integrations parameters ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: MODULES BASICS

Bade project structures ko organize karne aur variables conflict (global pollution) ko prevent karne ke liye script modules design kiye jate hain.

1. Modules Kya Hote Hain?

What is it? Module ek isolated single file script hoti hai jisme likhe code variables, functions automatic level par internal private boundaries share karte hain. Doosri files tabhi access kar sakti hain jab unhe explicitly **export** aur **import** kiya jaye.

Why JavaScript modules are needed? Global variables name collision errors ko stop rules apply karne ke liye. Bina modules ke, scripts browser level global window workspace share karti hain, jisse error patterns coordinate crashes badhte hain.

Normal JS file vs Module file:

- **Normal JS File:** Variables automatically enter global window space. Cannot use import / export.
- **Module File:** Local scope is isolated. Requires type="module" script loading.

2. export & import Methods

Two ways of sharing files:

Named Export

Share multiple components with their exact name identifiers. Imports must use curly braces {}. export const x = 10; → import { x } from "./module.js";

Default Export

Fallback export of the module. Only 1 default item can exist per file. Imports can skip curly braces {} and choose custom names. `export default function run() {}` → `import customRunner from "./module.js";`

PART 2: MULTI-FILE CODE SHARING EXAMPLES

Check how `app.js` imports values from `math.js`:

Inside `math.js`:

javascript

```
export const PI = 3.14;
```

```
export default function multiply(a, b) {
```

```
    return a * b;
```

```
}
```

Inside `app.js`:

javascript

```
import multiply, { PI } from "./math.js";
```

```
console.log(PI);
```

```
console.log(multiply(2, 3));
```

Expected Output 3.14 6

PART 3: ARCHITECTURE & `type="module"` HTML SETUP

1. Module Architecture

Typical modular layout:

text

project/

├── index.html (Loads script type="module")

└── src/

├── app.js (main entry point)

└── math.js (utility module helper)

2. script type="module" HTML Setup

HTML files must declare type="module" to notify browsers to run ESM (ES Modules) rules.

Example

html

```
<script type="module" src="app.js"></script>
```

Important point: Modules run in "use strict" mode automatically and defer execution by default.

QUICK REVISION SUMMARY

- **Modules:** File-scoped isolated JS scripts.
 - **Named:** Multiple exports, requires curly brackets {} in import, names must match.
 - **Default:** Single fallback export per file, can be renamed, curly brackets {} are not used.
 - **HTML integration:** Requires type="module" attribute configuration.
 - **Parsing:** Modules are compiled in strict mode and loaded asynchronously.
-

IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Strict Isolation:** Module scope boundaries rendering inner variables inaccessible to outside scripts unless exported.
 2. **ES Modules (ESM):** Native ES6 modules framework utilizing import and export statements.
 3. **Default Fallback:** The primary exported module slot allowing flexible client-side renaming.
 4. **Scope Pollution:** Namespace collisions occurring when scripts write directly to global scopes.
 5. **Dynamic Imports:** Asynchronous module loaders returning promises to load modules on demand.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. What is the difference between Named Export and Default Export?

Ans:

- Named exports allow exporting multiple items from a file, require exact matching names during import, and must be wrapped in curly braces {}.
- Default export allows only one item per file, can be imported without curly braces {}, and can be assigned a custom name during import.

Q2. Why do we need type="module" in HTML script tags?

Ans: Without type="module", the browser parses the file as a normal script, where import and export statements trigger SyntaxErrors. Adding the attribute tells the browser to parse it as an ES Module.

Q3. Do modules support hoisting?

Ans: Import statements are hoisted. JavaScript parses and resolves all imports at the top of the scope before executing any module code.

Q4. Difference between normal script files and module files?

Ans: Normal script files share variables globally, pollute the scope, and do not support import/export. Module files have file-level scope, run in strict mode by default, and support modular import/export structures.

Q5. Can we use multiple export default in a single file?

Ans: No. A module file can have at most one default export.

Q6. Predict output:

```
javascript
// Inside math.js
export default function test() { return 10; }

// Inside app.js
import myCustomName from "./math.js";
console.log(myCustomName());
```

Ans: 10. (Default export can be imported using any custom name).

Q7. How do you import all named exports from a file under a single namespace alias?

Ans: By using import * as AliasName from "./file.js".

Q8. Predict output:

javascript

// Inside helper.js

export const val = 5;

// Inside app.js

import { val as myVal } from "./helper.js"; // aliasing named export

console.log(myVal);

Ans: 5.

Q9. Can we use import inside standard loops or if conditions?

Ans: Standard static import statements must be placed at the top level of the file and cannot reside inside blocks. Dynamic imports use `import("path")` returning promises.

Q10. What does "use strict" do in modules?

Ans: Modules run in strict mode by default. It prevents mistakes like assigning values to undeclared variables.

Q11. Predict output of importing a missing named export:

Ans: `SyntaxError`: The requested module does not provide an export named 'X'.

Q12. Are modules deferred by default?

Ans: Yes. Modules are executed only after the HTML parser has completely parsed the document, equivalent to the `defer` attribute.

Q13. Predict output:

javascript

// Inside module.js

let x = 10; // no export

// Inside app.js

import "./module.js";

console.log(typeof x);

Ans: "undefined". (Variables in modules are file-scoped and not globally shared unless exported).

Q14. Can you export a variable after declaring it on a separate line?

Ans: Yes. E.g. `const x = 5; export { x };`

Q15. Can you mix named and default exports in a single file?

Ans: Yes. A module can have one default export along with multiple named exports.

Q16. Predict output:

```
javascript
```

```
import multiply, { PI } from "./math.js";
```

Ans: Loads default export as multiply and named export as PI successfully.

Q17. What is CommonJS?

Ans: The module specification system historically used in Node.js utilizing module.exports and require(). Modern code bases prefer ES Modules.

Q18. Predict output:

```
javascript
```

```
// Inside app.js
```

```
this.name = "Test";
```

```
console.log(this); // inside a module file
```

Ans: undefined. (At the top level of a module, this is undefined instead of the global window object).

Q19. What is the scope of variables declared inside an ES module?

Ans: Module scope. They are local to the file and invisible to other files unless exported.

Q20. Can we run ES Modules directly from local file protocol (double click index.html)?

Ans: Browsers block local file protocol ES Module imports due to CORS security policies. You must run them using a local development server (like VS Code Live Server).



PRACTICE QUESTIONS

Question 1: Basic Named Export

- **Question:** Create a file exporting variable `const appVersion = "1.0.0"` as named export, and import it.
- **Solution:**

```
javascript
```

```
// config.js
```

```
export const appVersion = "1.0.0";  
  
// app.js  
  
import { appVersion } from "./config.js";  
  
console.log("Version:", appVersion);
```

- **Explanation:** Named exports are written using export and imported using matching keys in curly braces {}.

Question 2: Default exports setup

- **Question:** Export a utility class Database as default export and load it under a custom name.
- **Solution:**

```
javascript  
  
// db.js  
  
export default class Database {  
    connect() { return "Connected"; }  
}  
  
// app.js  
  
import DbConnection from "./db.js";  
  
let db = new DbConnection();  
  
console.log(db.connect());
```

- **Explanation:** Default exports are imported without curly braces and can be renamed freely.

Question 3: Dynamic rename of Named Import

- **Question:** Import named function fetchUser and alias rename it to getUserData in import statement.
- **Solution:**

```
javascript  
  
// api.js  
  
export function fetchUser() { return "userObj"; }  
  
// app.js
```

```
import { fetchUser as getUserData } from "./api.js";
```

```
console.log(getUserData());
```

- **Explanation:** Renames imports inline using as keyword to prevent local scope name collisions.

10:18 PM